

蚂蚁AI/前端开发

一面

请简述 JavaScript 的数据类型，以及如何准确判断一个变量是否为数组？

在 AI 对话场景中，经常用到流式输出，请说明 SSE 和 WebSocket 的区别？

请手写一个 Promise.all 的实现。

在处理 AI 联想输入时，防抖（Debounce）和节流（Throttle）有什么区别？

CSS 如何实现一个类似 ChatGPT 的打字机光标闪烁效果？

解释一下 TypeScript 中的泛型（Generics），并写一个处理 AI 接口响应的定义。

浏览器跨域的常用解决手段有哪些？在 AI 调试阶段你通常怎么处理？

介绍一下 Fetch API 中的 ReadableStream，它在处理 LLM 响应时有什么作用？

二面（深入原理与项目） · 题目1-8

在 React/Vue 中设计一个 AI 聊天组件，你会如何考虑组件的抽象与性能？

AI 输出的长文本通常是 Markdown 格式，流式渲染时如何处理被截断的标签？

谈谈 RAG（检索增强生成）流程中，前端可以参与哪些优化工作？

什么是 Prompt Injection（提示词注入）？前端如何配合防范？

你们项目中是如何优化 AI 响应首屏耗时的？

介绍一下你对 AI Agent（智能体）的理解，前端如何管理 Agent 的状态？

在 Webpack 或 Vite 构建流程中，如何利用 AI 提升开发效率？

如果让你实现一个类似 GitHub Copilot 的 Web 版编辑器，你会选择哪些核心技术栈？

三面（架构与综合能力） · 题目1-5

蚂蚁内部有很多 AI 应用场景，如果让你设计一套通用的“AI 前端中台”架构，你会包含哪些模块？

谈谈你对“AI 驱动的前端开发模式（LCNC + AI）”未来趋势的看法。

在处理大规模 AI 交互数据时，如何设计前端的监控与异常告警系统？

聊聊你在过去一年中遇到的最具有挑战性的 AI 前端问题，你是如何解决的？

如何看待当前前端领域各种 AI 框架（如 LangChain.js, Vercel AI SDK）？选型时你会考虑哪些因素？

一面（技术基础）

1. 请简述 JavaScript 的数据类型，以及如何准确判断一个变量是否为数组？

难度：★

考点：JS 基础类型、类型检测

答案要点：

JavaScript 分为基本数据类型和引用数据类型，判断数组最可靠的方法是使用 `Array.isArray` 静态方法。

- 基本类型：String, Number, Boolean, Null, Undefined, Symbol, BigInt
- 引用类型：Object, Array, Function, Date, RegExp 等
- 判断数组：`Array.isArray()` 是首选，也可使用 `Object.prototype.toString.call()`
- 区别：`typeof` 对数组会返回 object，`instanceof` 在多 iframe 环境下可能失效

常见坑：

- 认为 `typeof` 可以区分数组和对象
- 忽略了 null 会被 `typeof` 识别为 object 的历史遗留问题

追问：

- 为什么 `typeof null` 是 object?
- 如何判断一个对象是否是“纯粹”的对象 (Plain Object) ?

2. 在 AI 对话场景中，经常用到流式输出，请说明 SSE 和 WebSocket 的区别？

难度：★★

考点：网络协议、流式传输

答案要点：

SSE 是基于 HTTP 的单向推送协议，而 WebSocket 是全双工的二进制协议，AI 问答通常优先选择 SSE。

- 协议属性：SSE 是应用层 HTTP 协议，WebSocket 是独立的 TCP 协议
- 数据流向：SSE 仅支持服务端向客户端推送，WebSocket 支持双向实时通信
- 断线重连：SSE 内置断线重连机制，WebSocket 需要开发者手动实现
- 传输效率：SSE 传输文本更轻量，WebSocket 适合高频交互或二进制传输

常见坑：

- 认为 SSE 只能传输文本，其实可以通过 Base64 传二进制
- 忽略了 HTTP/1.1 下 SSE 有 6 个连接数的浏览器限制

追问：

- 如果要在 SSE 中实现客户端中断请求，前端应该怎么做？

3. 请手写一个 Promise.all 的实现。

难度：★★

考点：异步编程、Promise 原理

答案要点：

Promise.all 接收一个 Promise 数组，只有当所有 Promise 都成功时才返回结果数组，只要有一个失败就立即拒绝。

- 计数器机制：维护一个 count 变量记录已完成的数量
- 结果顺序：通过索引确保输出结果的顺序与输入数组一致
- 异常处理：任何一个 Promise 失败，直接调用 reject
- 类型兼容：输入可能包含非 Promise 值，需用 Promise.resolve 包裹

常见坑：

- 结果数组的顺序写成了完成的先后顺序，而不是输入的索引顺序
- 忘记处理输入为空数组的情况

追问：

- Promise.allSettled 和 Promise.all 的区别是什么？

代码块

```
1  function promiseAll(promises) {
2    return new Promise((resolve, reject) => {
3      if (!Array.isArray(promises)) return reject(new TypeError('Arguments must
be an array'));
4      let resolvedCount = 0;
5      const results = [];
6      const len = promises.length;
7      if (len === 0) return resolve([]);
8      for (let i = 0; i < len; i++) {
9        Promise.resolve(promises[i]).then(res => {
10          results[i] = res;
11          resolvedCount++;
12          if (resolvedCount === len) resolve(results);
13        }).catch(reject);
14      }
15    });
16  }
```

4. 在处理 AI 联想输入时，防抖（Debounce）和节流（Throttle）有什么区别？

难度：★

考点：性能优化、函数高阶应用

答案要点：

防抖是在事件触发后延迟执行，若期间再次触发则重新计时；节流是规定时间内只执行一次。

- 防抖场景：搜索框输入联想、窗口大小调整（resize）
- 节流场景：滚动加载（scroll）、模型生成过程中的进度更新

- 核心原理：防抖利用 `clearTimeout` 重新计时，节流利用时间戳或定时器锁死执行频率
- 用户体验：防抖关注最后一次操作，节流关注执行过程的平滑度

常见坑：

- 在 React 组件中使用防抖时，没有处理组件卸载导致的内存泄漏
- 忘记处理 `this` 指向和参数传递

追问：

- 如何实现一个带“立即执行”功能的防抖函数？

5. CSS 如何实现一个类似 ChatGPT 的打字机光标闪烁效果？

难度：★

考点：CSS 动画、伪元素

答案要点：

通过 CSS 的 `animation` 配合 `border-right` 或伪元素，改变颜色透明度来实现闪烁感。

- 关键帧：使用 `@keyframes` 定义 `opacity` 从 0 到 1 的变化
- 布局：在文本容器后添加一个宽度极窄的 `inline-block` 元素
- 动画属性：设置 `step-end` 或 `linear` 循环播放
- 交互：当 AI 停止输出时，通过 JS 移除该类名或隐藏元素

常见坑：

- 闪烁频率过快导致视觉疲劳
- 忽略了文本换行时光标的位置对齐问题

追问：

- 如果实现文字逐个弹出的动画，不使用 JS 怎么做？

6. 解释一下 TypeScript 中的泛型（Generics），并写一个处理 AI 接口响应的定义。

难度：★★

考点：TS 进阶语法

答案要点：

泛型是指在定义函数或类时不预先指定具体类型，而是在使用时才指定类型的一种特性。

- 灵活性：允许代码复用，同时保持类型安全
- 约束：可以使用 `extends` 关键字对泛型进行约束
- 默认值：可以为泛型指定默认类型
- 实际应用：定义通用的 API 返回体结构

常见坑：

- 过度使用 `any` 导致泛型失去意义

- 复杂嵌套泛型导致代码可读性下降

追问：

- keyof 和 typeof 在泛型中怎么配合使用？

代码块

```
1  interface ApiResponse<T> {
2    code: number;
3    data: T;
4    message: string;
5  }
6
7  interface ChatMessage {
8    role: 'user' | 'assistant';
9    content: string;
10 }
11
12 async function fetchChat<T>(params: object): Promise<ApiResponse<T>> {
13   const response = await fetch('/api/chat', { method: 'POST', body:
JSON.stringify(params) });
14   return response.json();
15 }
```

7. 浏览器跨域的常用解决手段有哪些？在 AI 调试阶段你通常怎么处理？

难度：★

考点：浏览器安全、工程化

答案要点：

跨域是由同源策略引起的，常用的解决方法包括 CORS、反向代理和 JSONP。

- CORS：服务端设置 Access-Control-Allow-Origin 等响应头，是最标准的做法
- Proxy：在 Vite/Webpack 开发服务器中配置 proxy 转发请求到 AI 后端
- Nginx：在生产环境通过 Nginx 配置 location 转发
- 调试技巧：使用浏览器插件禁用同源策略（仅限本地）或配置本地 hosts

常见坑：

- 认为设置了 CORS 就不需要处理 Cookie（需设置 Allow-Credentials）
- 区分不清简单请求和预检请求（OPTIONS）

追问：

- 什么是简单请求？为什么 AI 接口通常会触发 OPTIONS 请求？

8. 介绍一下 Fetch API 中的 ReadableStream，它在处理 LLM 响应时有什么作用？

难度：★★★

考点：流式处理、前端底层 API

答案要点：

ReadableStream 允许在数据全部到达前逐块读取内容，是实现 AI 逐字生成效果的核心。

- 异步迭代：通过 `response.body.getReader()` 获取读取器
- 内存优化：不需要等待整个 JSON 加载完毕，降低了长文本下的内存占用
- 实时性：读取到一块数据（chunk）后立即通过 `TextDecoder` 解码并渲染
- 状态控制：可以随时调用 `reader.cancel()` 停止接收模型输出

常见坑：

- 忘记处理 `TextDecoder` 跨 chunk 解码导致的乱码问题（字节截断）
- 没有正确处理流关闭或异常中断的逻辑

追问：

- 如果后端返回的是 `EventStream` 格式，前端如何解析？

二面（深入原理与项目）

1. 在 React/Vue 中设计一个 AI 聊天组件，你会如何考虑组件的抽象与性能？

难度：★★

考点：组件设计、性能优化

答案要点：

AI 聊天组件需要兼顾消息列表的长列表渲染、Markdown 解析性能以及自动滚动交互。

- 状态管理：区分历史消息和当前正在生成的流式消息，避免频繁重绘整个列表
- 虚拟列表：当对话达到数百条时，使用虚拟滚动（Virtual List）保证流畅度
- 渲染优化：对已完成的消息使用 `memo` 或 `keep-alive`，仅更新正在生成的最后一条
- 交互细节：实现“用户手动滚动时停止自动置底”的逻辑，提升体验

常见坑：

- 每次流式更新都导致整个父组件重新渲染
- Markdown 渲染器在流式输出时频繁销毁重建导致闪烁

追问：

- 如何处理消息中包含的复杂数学公式（MathJax）或代码高亮？

2. AI 输出的长文本通常是 Markdown 格式，流式渲染时如何处理被截断的标签？

难度：★★★

考点：字符串处理、渲染策略

答案要点：

流式输出会导致 Markdown 语法（如代码块、加粗）在传输中途被截断，渲染引擎需要具备容错能力。

- 增量渲染：使用支持流式解析的 Markdown 库（如 markdown-it 的插件或自定义逻辑）
- 自动补全：在渲染未完成的 chunk 时，临时闭合未结束的代码块标签
- 缓冲区：设置微小的缓冲区，确保收集到完整的语法标识符后再渲染
- 状态机：通过状态机记录当前是否处于代码块或列表模式中

常见坑：

- 截断的代码块导致后续所有文字都变成了代码样式
- 频繁的 innerHTML 操作引起性能问题

追问：

- 如果 AI 输出的是 JSON 字符串流，如何实时解析其中的某个字段？

3. 谈谈 RAG（检索增强生成）流程中，前端可以参与哪些优化工作？

难度：★★★

考点：AI 业务理解、前端工程化

答案要点：

前端在 RAG 中不仅是展示层，还负责文档预处理、分段展示以及引用来源的溯源交互。

- 文档解析：在上传端进行 PDF/Markdown 的前端解析与清洗，减轻后端压力
- 引用溯源：实现点击 AI 回答中的引用标记，自动定位并高亮原始文档片段
- 语义搜索交互：设计多模态输入（图片、文件）的预览与管理界面
- 反馈闭环：收集用户对检索结果的点赞/踩，用于微调 Embedding 或 Prompt

常见坑：

- 溯源高亮时，前端与后端对文档偏移量的计算不一致
- 忽略了长文档在前端展示时的性能问题

追问：

- 如何在前端实现一个简单的向量相似度计算（如余弦相似度）？

4. 什么是 Prompt Injection（提示词注入）？前端如何配合防范？

难度：★★

考点：AI 安全、前端安全

答案要点：

提示词注入是指用户通过输入特殊指令误导模型执行非预期操作，前端是第一道防线。

- 输入过滤：在前端对敏感词、系统指令前缀（如 "Ignore previous instructions"）进行拦截
- 结构化输入：避免直接拼接用户字符串，通过 UI 引导用户输入特定参数

- 输出转义：对 AI 返回的内容进行严格的 XSS 过滤，防止模型生成恶意脚本
- 权限隔离：确保 AI 生成的内容无法直接调用具有高权限的 JS API

常见坑：

- 认为后端做了过滤前端就不需要管
- 直接将 AI 返回的 HTML 字符串用 v-html 或 dangerouslySetInnerHTML 渲染

追问：

- 如果 AI 具备调用插件的能力（Function Calling），前端如何保证调用安全？

5. 你们项目中是如何优化 AI 响应首屏耗时的？

难度：★★

考点：性能优化、用户体验

答案要点：

AI 接口响应慢是行业痛点，优化核心在于“感知提速”和“链路优化”。

- 骨架屏与占位：在等待模型响应时，展示具有 AI 特色的 Loading 动画或预热文本
- 接口并发：并行请求用户信息、上下文配置和 AI 接口
- 边缘计算：利用 Edge Function 处理简单的 Prompt 拼接，减少往返时延
- 缓存策略：对高频重复问题（如“你好”）在前端或 Service Worker 做语义缓存

常见坑：

- 所有的逻辑都堆在主线程，导致 Loading 动画卡顿
- 忽略了移动端网络不稳导致的流式传输中断

追问：

- 如何衡量 AI 产品的“首字响应时间（TTFT）”？

6. 介绍一下你对 AI Agent（智能体）的理解，前端如何管理 Agent 的状态？

难度：★★★

考点：Agent 架构、状态管理

答案要点：

Agent 是具有自主规划和工具调用能力的 AI 实体，前端需要管理其复杂的任务链状态。

- 任务拆解展示：将 Agent 的思考过程（Thought）、行动（Action）和观察（Observation）可视化
- 中间态维护：使用 Redux 或 Pinia 管理长周期的任务流，支持断点续传
- 交互干预：在 Agent 执行关键操作（如删除数据）前，前端弹出确认拦截
- 历史回溯：记录 Agent 的每一步决策快照，方便用户回滚或查看详情

常见坑：

- 状态机设计过于简单，无法处理 Agent 的并行任务

- 忽略了多 Agent 协作时的消息时序混乱问题

追问：

- 如果 Agent 执行时间超过 1 分钟，前端如何保持连接不超时？

7. 在 Webpack 或 Vite 构建流程中，如何利用 AI 提升开发效率？

难度：★★

考点：工程化、AI 提效

答案要点：

可以通过编写构建插件，在编译阶段接入大模型进行代码分析、注释生成或自动国际化。

- 自动注释：在构建时提取导出的函数，调用 AI 生成 TSDoc 注释
- 异常诊断：结合构建错误日志，调用 AI 给出修复建议并展示在控制台
- 资源优化：利用 AI 对图片进行智能压缩或对冗余代码进行语义化清理
- 自动化测试：根据组件结构自动生成单元测试用例（Vitest/Jest）

常见坑：

- 构建过程中频繁调用 AI 导致编译速度极慢（需增加缓存机制）
- 泄露公司核心代码到公网模型

追问：

- 如何在本地私有化部署一个辅助开发的 LLM？

8. 如果让你实现一个类似 GitHub Copilot 的 Web 版编辑器，你会选择哪些核心技术栈？

难度：★★★

考点：技术选型、复杂场景

答案要点：

这需要集成高性能编辑器内核、流式通信以及复杂的代码上下文感知算法。

- 编辑器内核：首选 Monaco Editor（VS Code 同款），支持丰富的 LSP 协议
- 实时通信：使用 WebSocket 传输代码补全建议，保证低延迟
- 上下文提取：前端需要实时解析 AST（抽象语法树），提取当前光标相关的类、函数定义
- 渲染层：利用 Monaco 的 Inline Completions API 实现灰色虚字预览效果

常见坑：

- 频繁发送全量代码导致带宽浪费（应发送增量 diff）
- 补全内容与用户输入冲突时的竞争条件处理

追问：

- 如何在浏览器端实现轻量级的代码语法解析？

三面（架构与综合能力）

1. 蚂蚁内部有很多 AI 应用场景，如果让你设计一套通用的“AI 前端中台”架构，你会包含哪些模块？

难度：★★★

考点：架构设计、中台思维

答案要点：

AI 中台旨在屏蔽底层模型差异，为上层业务提供标准化的组件、SDK 和调试工具。

- 组件库层：提供流式对话、Prompt 模板配置、RAG 溯源、多模态输入等标准 UI
- SDK 层：封装统一的流式请求协议、自动重连、Token 统计、埋点监控
- 调试工具：内置 Prompt IDE，支持对比不同模型、不同参数下的输出效果
- 策略中心：集成敏感词过滤、幻觉检测、多语言翻译等中间件

常见坑：

- 架构设计过重，导致业务接入成本比自研还高
- 忽略了不同业务对 UI 自定义程度的高要求

追问：

- 如何解决中台组件在不同框架（React/Vue/小程序）下的复用问题？

2. 谈谈你对“AI 驱动的前端开发模式（LCNC + AI）”未来趋势的看法。

难度：★★★

考点：技术视野、行业思考

答案要点：

未来前端开发将从“手写代码”转向“意图驱动”，AI 将覆盖从需求评审到上线监控的全链路。

- 需求转化：自然语言直接生成低代码平台的 Schema 或 DSL
- 辅助编程：从简单的代码补全进化到整个模块、甚至整个项目的自动生成
- 质量保障：AI 自动发现逻辑漏洞、性能瓶颈并提供一键修复
- 角色转变：前端工程师将更多承担“系统架构师”和“Prompt 调优师”的角色

常见坑：

- 盲目乐观，认为 AI 能完全替代初级前端
- 忽视了 AI 生成代码的可维护性和安全性问题

追问：

- 你认为目前 AI 生成前端代码最大的瓶颈在哪里？

3. 在处理大规模 AI 交互数据时，如何设计前端的监控与异常告警系统？

难度：★★★

考点：稳定性保障、数据分析

答案要点：

AI 应用的监控除了传统的性能指标，更需关注模型质量、流式中断率和用户负反馈。

- 业务指标：统计 TTFT（首字时间）、每秒 Token 数、对话轮数分布
- 异常捕获：监控 SSE 断连、JSON 解析失败、模型内容违规拦截
- 质量监控：接入用户点赞/踩的数据，结合 Prompt 版本进行归因分析
- 性能关联：分析模型响应延迟与用户流失率的关系，推动后端优化

常见坑：

- 只监控了接口报错，忽略了模型输出为空或乱码的“软错误”
- 埋点数据量过大影响页面性能

追问：

- 如何在不侵入业务代码的情况下实现这些监控？

4. 聊聊你在过去一年中遇到的最具有挑战性的 AI 前端问题，你是如何解决的？

难度：★★★

考点：问题解决能力、项目深度

答案要点：

（此题为开放题，需结合实际经验，建议按 STAR 原则回答）

- 背景：描述一个复杂的 AI 交互场景（如多 Agent 协同、超长文本渲染、复杂可视化集成）
- 冲突：遇到了什么技术瓶颈（如内存溢出、卡顿、交互冲突、模型幻觉导致 UI 崩溃）
- 行动：采取了哪些技术手段（如引入 Web Worker、优化渲染策略、设计复杂的缓存机制）
- 结果：量化的提升（如 FPS 从 30 提升到 60，用户满意度提升等）

常见坑：

- 描述过于平淡，体现不出“挑战性”
- 只说结论，说不清底层原理和推导过程

追问：

- 如果现在让你重新做这个项目，有哪些地方可以改进？

5. 如何看待当前前端领域各种 AI 框架（如 LangChain.js, Vercel AI SDK）？选型时你会考虑哪些因素？

难度：★★

考点：技术选型、生态认知

答案要点：

框架选型应基于业务复杂度、团队技术栈兼容性以及长期维护成本。

- 功能完备性：是否支持流式输出、缓存管理、Function Calling、RAG 集成

- 生态适配：对 React/Next.js 等主流框架的支持程度，是否有现成的 UI 组件
- 扩展性：是否方便自定义中间件、更换底层模型 Provider
- 性能开销：框架本身的体积、对首屏加载的影响

常见坑：

- 盲目追新，引入了处于早期阶段、API 频繁变动的库
- 忽略了框架对国内特定模型（如通义千问、文心一言）的适配成本

追问：

- 你更倾向于自研一套轻量级 SDK 还是集成第三方成熟框架？为什么？